AI WORKOUT COACH

# IMPLEMENTATION PLAN

Three weeks, one validation per week. Every step is small, tested on its own and confirmed before the next one starts. Written to be followed step by step in Cursor.

Week 1 · steps 1.1–1.6

## DATA PIPELINE + MODEL 1

Environment, capture decision, recording starts, Spark sync to Parquet, unified skeleton, Model 1 benchmark, rep counting.

Week 2 · steps 2.1–2.6

## MODEL 2 + LIVE CAPTURE

EC3D rep dataset, ST-GCN vs baselines, Kafka streaming of pose and heart rate, our videos through Models 1–2, Spark history job.

Week 3 · steps 3.1–3.5

## MODEL 3 + COACH + RESULTS

Fatigue and risk labels, Model 3 vs fixed-threshold rule (RQ2, RQ3), live coach with voice, RQ1–RQ4 evaluation, final report.

**How to use it.** Give Cursor `docs/PLAN.md` (this document's source) and the rules file `.cursor/rules/awc.mdc`. Ask for one step at a time: "Do step 1.3". Each step ends with tests, a README update and a commit. Steps marked **MUST** are required; **SHOULD** can be cut if only two weeks remain (section 4).

# 0 · HOW TO USE THIS PLAN

## 0.1 RULES FOR EVERY STEP (CURSOR MUST FOLLOW THESE)

1. **Read before writing.** Read `README.md`, `src/awc/config.py` and the files listed in the step before changing anything.
2. **Paths come from `src/awc/config.py`.** Never hard-code a path. Add new paths to `config.py`.
3. **Raw data is read-only.** Never write inside `data/raw/` except when recording our own sessions (step 1.2).
4. **No data or checkpoints in git.** They are already in `.gitignore`. Check `git status` before each commit.
5. **Every step ships with tests.** Unit tests use small synthetic arrays and need no dataset. Tests that need the real data are marked `@pytest.mark.data` and skip automatically when the data is missing.
6. **Every training run is logged** in `experiments/<YYYYMMDD>_<name>/` with `config.json`, `metrics.json`, `log.txt` and figures. Checkpoints go to `models/<same name>.pt`.
7. **Reproducible.** Fixed seed (42), fixed splits from `config.py`, git commit hash saved in `config.json`.
8. **Finish every step the same way:**
   - run `python -m pytest -q` (all green) and `python -m pytest -q -m data` (all green when data is present)
   - update the README: progress table + one dated log line with the key numbers
   - commit: `git commit -m "Step X.Y: <what was done>"`
9. **Keep it CPU-friendly.** We have no GPU (PyTorch 2.14 CPU). Models stay small; every training script has a `--quick` flag (tiny subset, 1 epoch) used by tests.
10. **Ask instead of guessing** when an instruction is ambiguous or a check fails twice.

## 0.2 ENVIRONMENT FACTS

| ITEM | VALUE |
|---|---|
| OS | Windows 11, Git Bash or PowerShell |
| Python | 3.12.10, PyTorch 2.14.1 **CPU** |
| Project path | `C:\Users\yassi\OneDrive\Bureau\projet DL BIG DATA` : **contains spaces, always quote paths** |
| Inside OneDrive | Spark and Kafka temporary folders must live **outside** OneDrive (use `C:\awc_tmp\`) or OneDrive sync will lock files |
| Not installed yet | Java 17, PySpark, MediaPipe, OpenCV, Kafka (step 1.1 installs them) |
| Devices | Poco X3 Pro (camera), Samsung Galaxy Watch 7 (Wear OS) |
| Constraint | All tools free. No Kotlin / Android Studio / emulator. We train our own models; only the pose estimator is pretrained. |

## 0.3 DATA FACTS ALREADY VERIFIED

**MM-Fit** ( `data/raw/mm-fit/w00 … w20` )

- 21 sessions, 616 sets, 6,160 reps, 10 exercises + `non_activity` = 11 classes.
- Every stream has a `frame` column on the **30 fps video clock**: this is the sync key.
- Streams we use: `pose_2d` (2, frames, 19) = x/y, then [frame, 18 joints, COCO-18 order, pixels, **y points down**]; `sw_l_acc` , `sw_l_gyr` (≈100 Hz, rows `[frame, ts_ms, x, y, z]` ); `sw_l_hr` (≈1 Hz, rows `[frame, ts_ms, bpm]` ).
- Heart rate is **missing** in w05, w10, w18.
- Labels: one row per **set** ( `start_frame, end_frame, reps, exercise` ), no per-rep boundaries, no form label.
- **Official split** (from the authors' `train_multimodal_ar.py` ):
- train `01 02 03 04 06 07 08 16 17 18`
- val `14 15 19`
- test (seen people) `09 10 11`
- test (unseen people) `00 05 12 13 20` , **our main RQ1 test**

**EC3D** ( `data/raw/ec3d/data_3D.pickle` )

- `labels` (29,789, 5) = exercise, subject, mistake code, rep index, frame. `poses` (29,789, 3, 25) = x, y, **z up**, OpenPose BODY_25, already normalized (mid-hip at origin, trunk upright).
- 371 reps, 4 people (Hugues, Isinsu, Sena, Vidit).
- Codes: 1 correct, 2 feet too wide, 3 knees inward, 4 not low enough, 5 front bent, 6 knee passes toe, 7 arched back, 8 hunched back, 10 unknown (**exclude 10**).
- Per exercise: squat {1,2,3,4,5}, lunge {1,4,6}, plank {1,7,8}.
- Baseline already measured (random forest, leave-one-person-out): squat 53 %, lunge 82 %, plank 90 %. Authors' graph network: 90.9 %.

## 0.4 THE THREE WEEKS AT A GLANCE

| WEEK | THEME | STEPS | SHOWN AT VALIDATION |
|---|---|---|---|
| 1 | Data pipeline + Model 1, start recording | 1.1 → 1.6 | Spark job → Parquet, Model 1 benchmark (9 runs), rep counting, first own sessions recorded |
| 2 | Model 2 + live capture | 2.1 → 2.6 | ST-GCN vs baselines on EC3D, live pose + heart rate through Kafka, Models 1–2 on our own videos, Spark history job |
| 3 | Model 3 + live coach + evaluation | 3.1 → 3.5 | Model 3 vs fixed-threshold rule, RQ1–RQ4 results, live demo with voice |

**If only two weeks remain:** see section 4 at the end. Steps are marked **MUST** or **SHOULD**.

# WEEK 1 · DATA PIPELINE AND MODEL 1

## STEP 1.1 · ENVIRONMENT AND FIRST COMMIT MUST

**Goal.** Everyone can run the project from a clean clone; the first commit exists.

**Instructions**

1. Create a virtual environment and install dependencies:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
pip install -e .
```

2. Update `requirements.txt` to contain exactly (no versions pinned unless an install fails): `numpy pandas pyarrow scipy scikit-learn matplotlib torch opencv-python mediapipe pyspark kafka-python pyttsx3 pytest jupyter markdown`.
3. Install **Java 17** (free: Eclipse Temurin 17 JDK). Set `JAVA_HOME` and add `%JAVA_HOME%\bin` to `PATH`. Check: `java -version` shows 17.
4. Spark on Windows needs `winutils.exe` to write files:
   - download `winutils.exe` and `hadoop.dll` for Hadoop 3.3 (GitHub repo `cdarlint/winutils`) into `C:\hadoop\bin\`
   - set `HADOOP_HOME=C:\hadoop`, add `C:\hadoop\bin` to `PATH`
   - create `C:\awc_tmp\spark` and use it as Spark's local dir
5. Add to `config.py`: `TMP = Path("C:/awc_tmp")`, `SPARK_LOCAL_DIR = TMP / "spark"`, and the MM-Fit split:

```
MMFIT_SPLIT = {
   "train": ["w01","w02","w03","w04","w06","w07","w08","w16","w17","w18"],
   "val": ["w14","w15","w19"],
   "test_seen": ["w09","w10","w11"],
   "test_unseen": ["w00","w05","w12","w13","w20"],
}
MMFIT_CLASSES = ["squats","lunges","bicep_curls","situps","pushups","tricep_extensions",
                 "dumbbell_rows","jumping_jacks","dumbbell_shoulder_press",
                 "lateral_shoulder_raises","non_activity"]
```

6. Create `src/awc/spark.py` with `get_spark(app_name, cores="local[*]")` that builds a `SparkSession` with `spark.local.dir` = `SPARK_LOCAL_DIR`, `spark.sql.session.timeZone=UTC`, driver memory 4g.
7. Create `src/awc/training/tracking.py`:
   - `seed_everything(seed=42)` (python, numpy, torch)
   - `new_run(name, config) -> Path`: creates `experiments/<YYYYMMDD>_<name>/`, writes `config.json` including the git commit hash
   - `save_metrics(run_dir, metrics: dict)`
8. Register the pytest marker `data` in `pyproject.toml` and add `tests/conftest.py` that skips `data` tests when `data/raw/mm-fit` or `data/raw/ec3d/data_3D.pickle` is missing.

9. Mark the existing tests in `tests/test_data.py` with `@pytest.mark.data`.
10. Write `docs/decisions.md` (empty template: date, decision, options considered, why). Every important choice gets one entry.

**Tests** (`tests/test_env.py`)

- `test_imports`: numpy, pandas, torch, cv2, mediapipe, pyspark import without error.
- `test_spark_roundtrip`: `get_spark("test", "local[2]")` writes a 3-row DataFrame to Parquet in `C:\awc_tmp\test_parquet` and reads it back (3 rows).
- `test_tracking`: `new_run("unit", {...})` creates the folder with `config.json` containing `git_commit`; clean up afterwards.

**Acceptance**

- [ ] `java -version` → 17
- [ ] `python -m pytest -q` green, `python -m pytest -q -m data` green
- [ ] `git status` shows no `.npy`, `.pickle`, `.zip`, `data/raw` files
- [ ] First commit done: `Step 1.1: environment, Spark helper, run tracking`

---

## STEP 1.2 · CAPTURE CHOICE AND RECORDING PROTOCOL MUST · START RECORDING THIS WEEK

**Goal.** Decide how phone video and watch heart rate reach the PC **without writing a watch app**, and start recording our own sessions now. Model 3 needs many sessions per person, so recording cannot wait until week 3.

**Instructions**

1. **Phone camera → PC.** Install the free Android app *IP Webcam* on the Poco X3 Pro. It serves `http://<phone-ip>:8080/video`. On the PC, read it with `cv2.VideoCapture(url)`. Measure the real frame rate over 30 s (target ≥ 25 fps at 720p). Fallback: DroidCam (free) over USB.
2. **Watch heart rate → PC.** Test these options in order and keep the first one that passes the criteria. They are third-party apps, so check that the feature still exists and is free before relying on it:
   - (A) *Pulsoid* (Wear OS app + WebSocket API with a personal token)
   - (B) *HypeRate* (Wear OS app + WebSocket API)
   - (C) *Sensor Logger* with Wear OS companion (HTTP push or CSV export)
   - (D) **fallback:** Samsung Health / Health Connect export after the session, aligned on the PC clock with a start marker
   - **Pass criteria:** free, at least one value every 2 s, a timestamp per value, no custom watch code
   - **Bonus:** check whether the option also gives wrist accelerometer/gyroscope. If not, Model 1 runs on pose only in live mode (supported by the modality dropout in step 1.5).
3. Write the decision in `docs/decisions.md` (what was tested, what passed, latency measured).
4. Create `scripts/record_session.py` (runs on the PC during training):
   - arguments: `--user {yassine,partner} --session-id s01 --camera-url ... --hr-source {pulsoid,hyperate,sensorlogger,none}`
   - records video from the camera URL to `video.mp4` with OpenCV **and** writes `frames.csv` (`frame, pc_ts_ms`): every frame gets the PC timestamp, so no clock sync with the phone is needed
   - receives heart rate and writes `hr.csv` (`pc_ts_ms, bpm`) with the PC receive time

- keyboard during the session: `s` = set starts, `e` = set ends, then it asks: exercise, reps done, RPE 1–10, form label (`correct` or an EC3D mistake name, default `correct`); appends to `sets.csv` (`set, exercise, start_ms, end_ms, reps, rpe, form_label`)
- writes `meta.json` (user, date, phone position, camera angle, body weight optional)
- output folder: `data/raw/own_recordings/<user>/<YYYY-MM-DD>_<session-id>/`

5. Write `docs/recording_protocol.md` and follow it for every session:
   - **Phone:** about 45° in front of the person, hip height, 2–3 m, whole body visible. Mark the tripod position on the floor with tape; same spot every session.
   - **Normal session (≈ 30 min):** 5 min warm-up, then 3 sets of squats and 3 sets of lunges, each set taken to RPE 8–9 (hard but **safe**: bodyweight or light load, stop immediately on pain), 90 s rest, RPE typed after each set.
   - **Form session (once a week):** 8 reps correct + 8 reps of each EC3D mistake (squat: feet too wide, knees inward, not low enough, front bent; lunge: not low enough, knee passes toe), each labelled in `sets.csv`.
   - **Targets by the end of week 3:** ≥ 6 normal sessions and ≥ 2 form sessions **per person**.

### Tests

- `tests/test_record_session.py` (no camera needed): the CSV writers produce the right headers; `sets.csv` rows validate (RPE 1–10, known exercise, known form label); folder naming follows the pattern.
- Manual: a 2-minute dry run; `video.mp4` plays, `frames.csv` has ≈ fps × 120 rows, `hr.csv` has ≥ 60 rows.

### Acceptance

- [ ] Decision recorded in `docs/decisions.md`
- [ ] Dry run passes the manual check
- [ ] **At least 2 real sessions recorded per person by validation 1**
- [ ] Commit: `Step 1.2: capture decision, recorder script, recording protocol`

---

## STEP 1.3 · SYNC MM-FIT STREAMS WITH SPARK → PARQUET MUST

**Goal.** One synced table per session at 30 Hz: pose + watch IMU + heart rate + label, written as Parquet by a Spark job. This is the big data stage.

**Files:** `src/awc/features/sync.py`, `src/awc/bigdata/build_interim.py`, `tests/test_sync.py`

### Instructions

1. In `features/sync.py` write pure pandas/numpy functions (testable without Spark):
   - `pose_to_frame(pose_2d) -> DataFrame` with columns `frame, x0..x17, y0..y17`; flip y (`y = -y_pixel`) so **y points up** everywhere in the project.
   - `imu_to_frame(arr, prefix) -> DataFrame`: IMU rows already carry the `frame` index; group by frame and take the mean → `frame, {prefix}_x, _y, _z` (≈ 3.3 samples per frame).
   - `hr_to_frame(arr, frames) -> DataFrame`: linear interpolation of bpm on the given frames; `hr_mask = 1` where a real value exists within ±5 s (150 frames), else `hr = NaN, hr_mask = 0`. A session without an HR file → all `hr_mask = 0`.
   - `attach_labels(df, labels) -> df`: adds `exercise` (from the set containing the frame, else `non_activity`), `set_id` (−1 outside sets), `set_reps`.
   - `sync_session(session_id) -> DataFrame`: loads streams with `awc.data.mmfit`, starts from pose frames (the reference clock), left-joins IMU and HR on `frame`, attaches labels, adds `session` and `split` (from `MMFIT_SPLIT`, w-id not in any list → `"unused"`).

2. In `bigdata/build_interim.py`:
   - Spark job: `spark.createDataFrame([(s,) for s in sessions])` → `groupBy("session").applyInPandas(sync_fn, schema)` → write `data/interim/mmfit_synced/` partitioned by `session`.
   - Flags: `--cores local[4]`, `--no-spark` (runs the same `sync_session` with plain pandas, for debugging), `--sessions w00 w01` (subset).
   - At the end print and save `data/interim/mmfit_synced_summary.json`: rows per session, % frames with IMU, % with HR, rows per class, runtime.
3. **Big data measurement** (needed for the report): run with `local[1]`, `local[2]`, `local[4]` and save runtimes in `experiments/<date>_spark_scaling/metrics.json`.

**Tests** (`tests/test_sync.py`)

- `test_imu_to_frame_mean`: synthetic IMU with 3 rows on frame 10 → one row with the mean.
- `test_hr_interpolation_and_mask`: HR at frames 0 and 60 → value at 30 is the midpoint; frames > 150 away get mask 0.
- `test_attach_labels`: frames inside / outside a set get the right `exercise` and `set_id`.
- `test_pose_y_flipped`: head y > ankle y after conversion.
- `@data test_sync_w00`: rows = number of pose frames in w00 (63,918); no NaN in pose columns; `hr_mask` mean > 0.9; 30 distinct `set_id ≥ 0`.
- `@data test_session_without_hr`: w05 has `hr_mask == 0` everywhere.

**Acceptance**

- [ ] `data/interim/mmfit_synced/` has 21 session partitions, total ≈ 1.46 M rows
- [ ] Summary JSON written; scaling runtimes saved
- [ ] Tests green
- [ ] README updated · commit `Step 1.3: Spark job syncing MM-Fit streams to Parquet`

---

## STEP 1.4 · ONE SKELETON FOR ALL SOURCES + WINDOWS MUST

**Goal.** MM-Fit, EC3D and live MediaPipe skeletons all become the same normalized 18-joint 2D skeleton. Then cut 3-second windows for Model 1.

**Files**: `src/awc/features/skeleton.py`, `src/awc/features/windows.py`, `tests/test_skeleton.py`, `tests/test_windows.py`

**Instructions**

1. Canonical skeleton = **COCO-18 order** (`awc.data.mmfit.JOINTS_2D`), coordinates `(x right, y up)`.
2. Mappings in `skeleton.py`:
   - `EC3D_TO_COCO18 = [0,1,2,3,4,5,6,7,9,10,11,12,13,14,15,16,17,18]` (BODY_25 indices; BODY_25 index 8 is mid-hip and is dropped)
   - `MEDIAPIPE_TO_COCO18`: nose 0; **neck = mean of 11 and 12**; r_shoulder 12, r_elbow 14, r_wrist 16, l_shoulder 11, l_elbow 13, l_wrist 15, r_hip 24, r_knee 26, r_ankle 28, l_hip 23, l_knee 25, l_ankle 27, r_eye 5, l_eye 2, r_ear 8, l_ear 7. MediaPipe image y points down → flip.
   - `BONES` (17 pairs, same list as the presentation): used by the graph network in week 2.
3. `normalize(seq)` for `seq` of shape (T, 18, 2):
   - origin = mid-hip = mean of r_hip (8) and l_hip (11), per frame

- scale = **median over the sequence** of the neck→mid-hip distance (torso length); divide all coordinates by it
- return float32; must be invariant to translation and uniform scaling

4. `project_ec3d(p3d, yaw_deg)` for EC3D (T, 3, 25): find the facing direction from toes minus heels (same as `scripts/analysis/angle_experiment.py`), rotate by `yaw_deg`, keep horizontal coordinate as x and z as y → (T, 18, 2) via `EC3D_TO_COCO18`.
5. `mirror(seq)`: flip x and swap left/right joints (augmentation).
6. `windows.py`:
   - `make_windows(df, length=90, hop=45)` per session from the synced Parquet
   - outputs: `X_pose` (N, 90, 36) normalized, `X_imu` (N, 90, 6) = acc + gyr, `imu_mask` (N,), `y` (N,) = class index of the **center frame**, `session` (N,), `start_frame` (N,)
   - drop windows where pose has NaN; windows where IMU is missing keep `imu_mask = 0` and zeros
   - IMU standardization: mean/std computed **on the train split only**, saved to `data/processed/imu_stats.json`, applied to all splits
   - save `data/processed/m1_windows.npz`
   - print the count per split and class (expected ≈ 32,000 windows in total)

### Tests

- `test_normalize_invariance`: random skeleton, then translated and scaled ×2.5 → identical output (tolerance 1e-5).
- `test_mapping_lengths`: both mappings give 18 joints; MediaPipe neck = midpoint of shoulders.
- `test_project_front_vs_side`: a synthetic T-pose seen at 0° has wide shoulders; at 90° shoulder width ≈ 0.
- `test_mirror_twice_identity`.
- `test_window_count`: 300 synthetic frames, length 90, hop 45 → 5 windows; center labels correct.
- `@data test_windows_file`: npz exists, shapes consistent, no NaN, all 11 classes present in train.

### Acceptance

- [ ] `m1_windows.npz` and `imu_stats.json` written; counts printed and pasted in the README log
- [ ] Tests green · commit `Step 1.4: unified skeleton, EC3D projection, Model 1 windows`

---

## STEP 1.5 · MODEL 1: RECOGNITION BENCHMARK + REP COUNTING MUST

**Goal.** Train Model 1 (which exercise), compare 3 architectures × 3 input sets, and count reps inside each set. Answers **RQ1**.

**Files**: `src/awc/models/m1_recognition.py`, `src/awc/training/train_m1.py`, `src/awc/features/reps.py`, `src/awc/evaluation/metrics.py`, `tests/test_m1.py`, `tests/test_reps.py`

### Instructions

1. `m1_recognition.py`: three encoders with the same interface `forward(x: (B, T, C)) -> (B, D)`:
   - `CNN1DEncoder`: 3 × [Conv1d(k=5) → BatchNorm → ReLU → MaxPool(2)], channels 64-128-128, global average pool → 128
   - `LSTMEncoder`: 2-layer bidirectional LSTM, hidden 64, mean over time → 128
   - `TransformerEncoder`: linear to 64, learned positional embedding (90), 2 layers, 4 heads, dropout 0.1, mean pool → 64
   - `M1Model(arch, inputs)`: `inputs ∈ {pose, imu, both}`; one encoder per used modality; fusion = concat(pose_emb, imu_emb × imu_mask, imu_mask) → Linear 128 → ReLU → Dropout 0.3 → Linear 11

- **Modality dropout**: in training with `inputs=both`, set `imu_mask = 0` for 30 % of samples. This lets the same model run on pose only in live mode if the watch gives no IMU.

2. `train_m1.py` (argparse): `--arch {cnn,lstm,transformer} --inputs {pose,imu,both} --epochs 30 --batch 128 --lr 1e-3 --seed 42 --quick`
   - loss: cross-entropy with class weights (inverse frequency, train split)
   - early stopping on **validation macro-F1**, patience 5; keep the best checkpoint
   - evaluate on `test_unseen` (main) and `test_seen`
   - save run folder: metrics (accuracy, macro-F1, per-class F1 for both tests, train time, parameter count), `confusion_test_unseen.png`, checkpoint in `models/`
   - `--quick`: 512 train windows, 1 epoch (used by tests)
3. `scripts/run_m1_benchmark.py`: runs the 9 combinations and writes `reports/m1_benchmark.md` (table: arch × inputs → macro-F1 unseen, accuracy, params, train minutes).
4. `features/reps.py`, rep counting and boundaries inside a set (exercise-agnostic):
   - input: normalized pose of one set (T, 18, 2)
   - signal = first principal component of the 36 coordinates over the set (the main movement direction), smoothed with a moving average of 9 frames
   - `find_peaks` (scipy) with `distance = 0.6 s × 30` and `prominence = 0.3 × std(signal)`; also try the negated signal and keep the version whose count has the lower variance of peak-to-peak intervals
   - return `count`, `boundaries` = list of (start_frame, end_frame) per rep (valleys between peaks)
   - `scripts/eval_rep_counting.py`: run on every labelled MM-Fit set; table per exercise: exact %, within ±1 %, mean absolute error → `reports/rep_counting.md` + `experiments/<date>_rep_counting/metrics.json`
5. `evaluation/metrics.py`: `macro_f1`, `per_class_f1`, `confusion(y, p, labels)`, `plot_confusion(...)`, `rep_count_errors(...)`.

### Tests

- `test_encoders_shapes`: each encoder maps (4, 90, 36) to (4, D).
- `test_m1_forward_all_inputs`: `M1Model` returns (B, 11) for pose, imu, both; works with `imu_mask = 0`.
- `test_one_step_decreases_loss`: 10 optimizer steps on a fixed tiny batch reduce the loss.
- `test_train_quick`: `train_m1.py --quick` finishes and writes `metrics.json`.
- `test_rep_count_sine`: a synthetic skeleton moving with 10 sine cycles → count 10.
- `@data test_rep_count_mmfit_squats`: squat sets of w00 → mean absolute error ≤ 1.

### Acceptance

- [ ] 9 runs logged; `reports/m1_benchmark.md` written
- [ ] **Target:** best macro-F1 on `test_unseen` ≥ 0.85. Report the real number either way; if lower, write in the README what was tried.
- [ ] Rep counting MAE ≤ 0.5 rep per set averaged over exercises (report per exercise)
- [ ] Tests green · commit `Step 1.5: Model 1 benchmark and rep counting`

---

## STEP 1.6 · VALIDATION 1 PACKAGE MUST

### Instructions

1. `reports/week1.md` (one page): what was built, the Spark scaling numbers, the M1 benchmark table, rep-counting table, number of own sessions recorded, problems met and decisions taken.
2. Figures in `reports/figures/`: M1 confusion matrix (best model), rep counting error per exercise, Spark runtime vs cores.
3. README: steps 1.1–1.5 marked done with dates; log lines with key numbers.
4. Demo commands that work from a clean terminal:

```
python -m awc.bigdata.build_interim --sessions w00 --cores local[2]
python -m awc.training.train_m1 --arch cnn --inputs both --quick
python scripts/eval_rep_counting.py
```

**Acceptance**: the user confirms validation 1 content. Commit `Week 1 validation package`.

---

# WEEK 2 · MODEL 2 AND LIVE CAPTURE

## STEP 2.1 · EC3D REP DATASET FOR MODEL 2 MUST

**Goal.** Every EC3D rep becomes a fixed-size 2D skeleton sequence that looks like what our phone sees.

**Files**: `src/awc/features/form_dataset.py` , `tests/test_form_dataset.py`

### Instructions

1. Load with `awc.data.ec3d.load()` and `reps()` . Drop code 10 (unknown) → 362 reps.
2. For each rep: `project_ec3d` at a yaw angle → `normalize` → **resample to T = 64 frames** (linear interpolation over time).
3. Labels: `exercise ∈ {squat, lunge, plank}` and `mistake` = index inside that exercise (squat 5 classes, lunge 3, plank 3); also `is_correct` (binary).
4. Angles:
   - **train:** yaw ∈ {0, 15, 30, 45, 60, 75, 90} × {normal, mirrored}
   - **test:** 45° only (main) and the full sweep (robustness, reported separately)
5. Online augmentation (train only, in the PyTorch Dataset): scale ×U(0.9, 1.1), time-warp ×U(0.8, 1.2) then resample back to 64, Gaussian noise σ = 0.01.
6. Folds: **leave one person out** (4 folds). A person never appears in train and test of the same fold.
7. Save `data/processed/m2_reps.npz` (base 3D-projected arrays + metadata); angles and augmentation are applied in the Dataset so the file stays small.

### Tests

- `test_resample_length` : any input length → 64 frames; endpoints preserved.
- `test_no_subject_leak` : for each fold, train subjects ∩ test subjects = ∅.
- `test_label_spaces` : squat labels in 0..4, lunge 0..2, plank 0..2.
- `@data test_rep_counts` : 362 reps after dropping code 10; per exercise: squat 132 (141 minus 9 unknown), lunge 127, plank 103.

**Acceptance**: tests green · commit `Step 2.1: EC3D rep dataset with projections and LOSO folds` .

---

## STEP 2.2 · MODEL 2: ST-GCN VS LSTM VS RANDOM FOREST MUST

**Goal.** Judge each rep: correct or which mistake. Compare a graph network with two baselines.

**Files**: `src/awc/models/m2_form.py` , `src/awc/features/form_features.py` , `src/awc/training/train_m2.py` , `tests/test_m2.py`

### Instructions

1. `form_features.py` : move the 6 hand-made measurements from `scripts/analysis/extract_ec3d.py` here (knee width, feet width, depth, trunk-vs-shin angle, knee-past-toe, hip sag) and add 2D versions computed on the projected skeleton. The analysis script then imports them (no duplicated code).
2. `m2_form.py` :
   - adjacency `A` from `BONES` + self-loops, normalized `D^-1/2 A D^-1/2` (18 × 18)

- `STGCNBlock(c_in, c_out)`: spatial graph conv (`X·A·W`) → BatchNorm → ReLU → temporal Conv (kernel 9 over time) → BatchNorm → residual → ReLU
- `STGCN`: input (B, 2, 64, 18) → blocks 2→64, 64→64, 64→128 (stride 2 in time on the last) → global average pool → **one head per exercise** (5/3/3 outputs). The exercise is known (from Model 1), so each sample uses its exercise head.
- `AngleLSTM` baseline: per frame 9 joint angles (both knees, hips, elbows, shoulders + trunk lean) → 2-layer LSTM hidden 64 → per-exercise heads
- random forest baseline: `form_features` → sklearn `RandomForestClassifier(300)`

3. `train_m2.py --model {stgcn,lstm,rf} --epochs 60 --lr 1e-3 --quick`:
   - 4 LOSO folds; fixed epochs (no early stopping, too little data for a validation split)
   - metrics per fold and mean ± std: accuracy per exercise, macro-F1, **recall of "not correct"** (binary: any mistake vs correct), test at 45° and at every angle
   - outputs: run folder with metrics, a bar chart of accuracy per exercise for the 3 models, and a checkpoint trained on **all 4 people** (used live)
4. `reports/m2_benchmark.md`: table models × exercise (mean ± std), plus the angle robustness curve.

### Tests

- `test_adjacency_symmetric_normalized`
- `test_stgcn_forward`: (8, 2, 64, 18) + exercise ids → correct output size per exercise head
- `test_overfit_tiny`: 16 reps, 100 steps → train accuracy 100 %
- `test_train_quick` for stgcn and rf

### Acceptance

- [ ] All three models benchmarked with LOSO
- [ ] **Target:** ST-GCN beats the random forest on mean accuracy for squat (> 53 %); report all numbers honestly
- [ ] Recall of mistakes reported (missing a bad rep is the worst error)
- [ ] Commit `Step 2.2: Model 2 benchmark on EC3D`

---

## STEP 2.3 · KAFKA + LIVE PRODUCERS (POSE AND HEART RATE) MUST

**Goal.** Live streams from the phone and the watch reach the laptop through Kafka.

**Files**: `src/awc/streaming/schemas.py`, `pose_producer.py`, `hr_producer.py`, `monitor.py`, `sink.py`, `tests/test_streaming.py`, `docs/kafka_setup.md`

### Instructions

1. **Kafka setup** (`docs/kafka_setup.md`):
   - **Option A (simplest):** Docker Desktop (free for personal/education), `docker run -p 9092:9092 apache/kafka:latest`
   - **Option B:** Apache Kafka 4.x binaries in KRaft mode, using `bin\windows\*.bat`, with `log.dirs=C:/awc_tmp/kafka`
   - Write down which one works.
2. Topics: `pose.keypoints`, `watch.hr`, `watch.imu` (only if step 1.2 found an IMU source), `coach.events`. 1 partition each.
3. `schemas.py`: dataclasses + `to_json` / `from_json` + `validate`:

- `pose.keypoints`: `{"session", "frame", "pc_ts_ms", "kp": [[x, y, visibility] × 18]}` (already mapped to COCO-18, image coordinates; normalization happens in the consumer)
- `watch.hr`: `{"session", "pc_ts_ms", "bpm"}`
- `coach.events`: `{"session", "pc_ts_ms", "type": "rep|form|risk|say", "payload": {...}}`

4. `pose_producer.py --camera-url ... | --video file.mp4 --session ...`: OpenCV frames → MediaPipe Pose Landmarker (Tasks API, model `pose_landmarker_full.task`, CPU) → `MEDIAPIPE_TO_COCO18` → Kafka. With `--video` it **replays a recorded file at real speed**: this is the safety net for the demo.
5. `hr_producer.py --source {pulsoid,hyperate,sensorlogger,replay} --session ...`: the source chosen in step 1.2, or replay of an `hr.csv`.
6. `monitor.py`: consumes all topics and prints, every 5 s, messages per second per topic and median latency (`now − pc_ts_ms`).
7. `sink.py`: consumes topics and appends to Parquet in `data/raw/own_recordings/<user>/<session>/stream/`, so live sessions are also recorded.

### Tests

- `test_schema_roundtrip` for every message type; `validate` rejects wrong joint counts, missing fields, bpm outside 30–230.
- `test_mediapipe_mapping_on_image`: on a test image of a standing person in `tests/assets/` (take one photo), 18 joints are returned and the head is above the ankles.
- Integration (`@pytest.mark.kafka`, skipped when no broker): produce 100 pose messages, consume them in order, median latency < 200 ms.

### Acceptance

- [ ] Live: monitor shows ≈ 25–30 msg/s on `pose.keypoints` and ≈ 1 msg/s on `watch.hr` for 2 minutes
- [ ] Replay of a recorded session works with the broker
- [ ] Commit `Step 2.3: Kafka streaming of pose and heart rate`

---

## STEP 2.4 · OUR RECORDINGS THROUGH MODELS 1 AND 2 SHOULD · FIRST LOOK AT RQ4

**Goal.** Run the trained models on our own videos and compare with the dataset results.

**Files**: `scripts/process_recording.py`, `src/awc/data/recordings.py` (implement `load`), `tests/test_recordings.py`

> **Timing rule.** The clock of a recording is `frames.csv` (`pc_ts_ms` of every frame), never the video's fps: phone streams are uneven, and a wrong clock distorts rep durations, the main fatigue signal. Call `extract_pose(video, frame_times_ms=<frames.csv pc_ts_ms>)`, which resamples to 30 fps on the real times (`resample_times`, tested in `tests/test_pose_timing.py`). Set start/end in `sets.csv` are on the same clock.

### Instructions

1. `process_recording.py <session folder>`: `video.mp4` → MediaPipe → COCO-18 → normalize → `pose.parquet` (frame, pc_ts_ms, joints); join `hr.csv` by nearest PC timestamp (±2 s) → synced table like MM-Fit interim (same columns). Write it to `data/interim/own/<user>/<session>.parquet`.
2. `recordings.load(session_id)` returns that table plus `sets.csv`.

3. Model 1 on own sets: exercise accuracy (squats vs lunges vs non_activity, pose only, since we may not have the watch IMU) and rep counting MAE against the reps typed in `sets.csv`.
4. Model 2 on form sessions: accuracy per mistake against `form_label`.
5. `reports/rq4_first_look.md`: dataset vs own-device numbers side by side.

**Acceptance**: tables written, even if numbers are low (that is a valid RQ4 finding) · commit `Step 2.4: Models 1-2 on our recordings`.

---

## STEP 2.5 · SPARK HISTORY JOB: REP TABLE + PERSONAL BASELINES MUST

**Goal.** Build the per-rep history that Model 3 learns from, with Spark.

**Files**: `src/awc/bigdata/history.py`, `tests/test_history.py`

### Instructions

1. **Rep table**, one row per rep, from MM-Fit interim (squats, lunges, and every other exercise where rep counting works) and own interim. Columns:
   - `source, user, session, session_date, exercise, set_id, rep_idx, n_reps_in_set`
   - `duration_s, rom` (range of the main signal), `mean_speed = rom / duration`
   - `form_score` (Model 2 probability of "correct"; NaN for exercises Model 2 doesn't cover)
   - `hr_mean, hr_end, hr_rise` (hr_end − HR at set start), `hr_mask`
   - `rpe` (own data only)
   - For MM-Fit `user = session` (the session→person mapping is unknown; treat each session as its own history).
2. **Personal baselines** with Spark window functions:
   - `Window.partitionBy("user", "exercise").orderBy("session_date").rowsBetween(Window.unboundedPreceding, -1)`
   - **strictly past sessions, no leakage**
   - for each session: mean and std of the first-3-reps duration, ROM, speed and HR rise in previous sessions
   - first session of a user: baseline = NaN → flagged `has_baseline = 0`
3. Write `data/processed/rep_table.parquet` and `data/processed/baselines.parquet`; print the row counts.

### Tests

- `test_baseline_uses_only_past`: synthetic user with 3 sessions → session 3's baseline equals the mean of sessions 1–2, session 1's baseline is NaN.
- `test_rep_table_columns`: schema and dtypes.
- `@data`: rep table has > 5,000 MM-Fit rows.

**Acceptance**: both Parquet files exist; tests green · commit `Step 2.5: Spark rep history and personal baselines`.

---

## STEP 2.6 · VALIDATION 2 PACKAGE MUST

- `reports/week2.md`: M2 benchmark (vs RF baseline and vs the authors' 90.9 %), angle robustness, Kafka throughput and latency, RQ4 first look, number of own sessions so far.

- Live demo: `monitor.py` running with the phone and watch; then a replay.
- README updated · commit `Week 2 validation package`.

---

# WEEK 3 · MODEL 3, LIVE COACH, EVALUATION

## STEP 3.1 · MODEL 3 DATASET AND LABELS MUST

**Goal.** Turn the rep history into sequences with fatigue and risk labels.

**Files**: `src/awc/features/fatigue_dataset.py` , `tests/test_fatigue_dataset.py`

### Instructions

1. One sample = **one set** = a sequence of reps; per-rep features:
   - `duration, rom, mean_speed, form_score, hr_end, hr_rise, rep_idx / n, time since set start`
   - masks for missing `form_score` and HR
2. Two normalizations (this is the RQ3 comparison):
   - **general:** z-score with global train statistics
   - **personal:** z-score with the user's baseline from step 2.5 (fallback to global when `has_baseline = 0` )
3. Labels:
   - **fatigue score** (all data): `rep_idx / (n - 1)` = position in the set (weak label, 0 fresh → 1 tired)
   - **breakdown event** (own data, and MM-Fit as a weak version): the first rep where speed has dropped ≥ 20 % vs the mean of the first 2 reps **or** `form_score < 0.5` . Make the 20 % a parameter.
   - **risk label** per rep: 1 if a breakdown happens at this rep or within the next 2 reps
   - **RPE** per set (own data): used only for validation, never for training
4. Splits:
   - MM-Fit: same session split as Model 1
   - own data: **time-ordered per user**, the last 2 sessions of each person are the test set

### Tests

- `test_fatigue_label_range` : 0 to 1, increasing.
- `test_breakdown_detection` : synthetic speeds [1, 1, 0.95, 0.85, 0.78] with a 20 % threshold → breakdown at index 4.
- `test_personal_norm_fallback` : no baseline → global stats used.
- `test_time_split` : test sessions are later than all train sessions of the same user.

**Acceptance**: dataset built, counts printed · commit `Step 3.1: Model 3 sequences and labels` .

---

## STEP 3.2 · MODEL 3 TRAINING AND THE RQ2 / RQ3 COMPARISONS MUST · OUR CONTRIBUTION

**Files**: `src/awc/models/m3_fatigue.py` , `src/awc/training/train_m3.py` , `tests/test_m3.py`

### Instructions

1. `M3Model` : 1-layer GRU (hidden 32) over reps → two heads per rep: `fatigue` (sigmoid, MSE loss) and `risk` (sigmoid, BCE loss with positive weight). Small on purpose: little data.
2. Training: pre-train on MM-Fit (fatigue + weak risk), then fine-tune on own train sessions. `--norm {general,personal} --hr {on,off}` .

3. Compare **four variants plus one rule** (see the table after this list).
4. Metrics on the own test sessions (and MM-Fit test for the weak version):
   - risk AUC
   - **warning lead time**: reps between the first alert (risk ≥ 0.7) and the breakdown; alerts after the breakdown count as 0
   - **false alarms per session**: alerts in sets with no breakdown
   - **Spearman correlation** between the predicted fatigue of the last rep of each set and the RPE typed for that set
5. `reports/m3_results.md` : table of the 5 variants; **RQ2 = B4 vs B3**, **RQ3 = B4 vs B2**, existing apps = B4 vs B1. Per-person numbers too (only 2 people: say so).

| VARIANT | WHAT | ANSWERS |
|---|---|---|
| B1 rule | stop when speed loss > 25 % (like the Metric VBT app) | "existing apps" |
| B2 | general normalization, with HR | RQ3 reference |
| B3 | personal normalization, without HR | RQ2 reference |
| **B4 (ours)** | personal normalization, with HR | RQ2, RQ3 |
| B5 | general, without HR | completeness |

### Tests

- `test_m3_forward_variable_length` (padding + mask)
- `test_rule_baseline` : synthetic speeds trigger exactly at the 25 % point
- `test_lead_time_metric` : hand-made example with a known lead time
- `test_train_quick`

### Acceptance

- [ ] All variants trained and evaluated on the same test sessions
- [ ] Results reported honestly, including if B4 does not win (that is a valid finding)
- [ ] Commit `Step 3.2: Model 3 and RQ2/RQ3 comparisons`

---

## STEP 3.3 · LIVE COACH WITH VOICE SHOULD

**Goal.** Everything runs live: phone + watch → Kafka → Models 1-2-3 → spoken feedback.

**Files**: `src/awc/coach/live.py` , `src/awc/coach/policy.py` , `src/awc/coach/voice.py` , `tests/test_policy.py`

### Instructions

1. `live.py` consumes `pose.keypoints` and `watch.hr` :
   - ring buffer of 90 frames
   - Model 1 every 15 frames; exercise = majority over the last 4 predictions
   - online rep detection: the step 1.5 rep detector on a sliding buffer with 0.5 s lag
   - when a rep ends: rep tensor → Model 2 (if squat/lunge) → per-rep features with the personal baseline → Model 3 → `policy.decide`
2. `policy.py` :

- risk ≥ 0.7 → "Stop the set."
- risk 0.4–0.7 → "Slow down, control the movement."
- Model 2 mistake with probability ≥ 0.6 → the cue for that mistake (e.g. knees inward → "Push your knees out.")
- else "Good rep" spoken every 3 reps only
- never more than one message per rep

3. `voice.py` : `pyttsx3` (free, offline, uses Windows voices), speaking in a background thread so it never blocks inference.
4. Publish every decision on `coach.events` ; log latency (rep end → speech start).
5. Optional overlay window (OpenCV): skeleton, exercise, rep count, last message.
6. `--replay <session folder>` runs the whole chain on a recorded session: **the demo backup**.

**Tests**

- `test_policy_priorities` : stop beats form cue beats "good rep"; one message per rep.
- `test_policy_debounce` .
- Manual: replay of a recorded set → spoken messages appear at the right reps; latency logged.

**Acceptance**

- [ ] Live run of one set works end to end
- [ ] Median latency rep end → voice < 1 s
- [ ] Replay mode works without the phone and watch
- [ ] Commit `Step 3.3: live coach with voice feedback`

---

## STEP 3.4 · EVALUATION RQ1–RQ4 AND FINAL REPORT MUST

**Files**: `src/awc/evaluation/collect.py` , `reports/results.md` , `reports/final_report/`

**Instructions**

1. `collect.py` reads every `experiments/*/metrics.json` and writes `reports/results.md` :
   - **RQ1:** M1 macro-F1 on unseen people, best architecture, camera vs watch vs both (sanity check), rep counting
   - **RQ2:** B4 vs B3
   - **RQ3:** B4 vs B2, plus B4 vs the fixed rule B1
   - **RQ4:** dataset vs own devices (M1, M2, M3)
   - **Big data:** Spark scaling, Kafka throughput and latency
2. Figures: M1 confusion, M2 per-exercise bars, M3 lead time and false alarms per variant, Spark runtime vs cores.
3. Final report (HTML → PDF, same look as the briefing in `reports/briefing/` ), following the teacher's format: objective, research questions, context and existing apps, data, approach, results, limits, references.
4. Add a "Results" slide to the presentation appendix if asked.

**Acceptance**: every number in the report comes from a `metrics.json` file · commit `Step 3.4: evaluation and final report` .

## STEP 3.5 · FINAL VALIDATION REHEARSAL MUST

- Fresh clone test: clone the repo in another folder, follow the README setup, run `python -m pytest -q`: must pass (data tests skip).
- Run the demo twice: live, then replay.
- README: all steps done, final log line, final results table.
- Tag the release: `git tag v1.0`.

---

# 4 · IF ONLY TWO WEEKS REMAIN

| KEEP | CUT OR SIMPLIFY |
|---|---|
| Week 1 as written (1.1 → 1.6) | — |
| 2.1, 2.2 (Model 2) | 2.4 → only Model 1 rep counting on own videos |
| 2.3 in replay-only form (recorded files → Kafka) | live watch connection becomes optional |
| 2.5, 3.1, 3.2 on whatever own sessions exist | 3.3 live coach → replay demo + the presentation's replay slide |
| 3.4 results + short report | final report kept to the results and limits sections |

**Never cut:** the own recordings (1.2), the Spark jobs (1.3, 2.5), and Model 3 with the B1–B4 comparison (3.2). They carry the project's contribution and the big data part.

# 5 · CHECKLIST TEMPLATE TO PASTE AT THE END OF EVERY STEP

```
Step X.Y done
- [ ] python -m pytest -q          → all green
- [ ] python -m pytest -q -m data  → all green (data present)
- [ ] metrics.json saved in experiments/ (if training)
- [ ] README progress table + log line updated
- [ ] git status clean of data/checkpoints
- [ ] git commit -m "Step X.Y: ..."
- [ ] user confirmed → next step
```